

Design and Analysis of Algorithm – III UNIT

2 marks:

1. Define decrease and conquer technique and list any two of its variations.

Decrease and conquer is a problem-solving technique where a problem is solved by reducing it to a simpler or smaller instance of the same problem and then solving the simpler instance.

Variations:

- Decrease by a constant : In each step, the problem size is reduced by a fixed amount.
- Decrease by a constant factor : In each step, the problem size is reduced by a fixed fraction.

2. What is a decrease by a constant technique and give an example.

In the decrease-by-a-constant variation, the size of an instance is reduced by the same constant on each iteration of the algorithm. Typically, this constant is equal to one.

Insertion sort is a classic example of a decrease-by-constant algorithm. For each subsequent element, create a subproblem consisting of the sorted portion of the array (to its left) and the element to be inserted.

3. What is a decrease by a constant factor technique and give an example.

The decrease-by-a-constant-factor technique suggests reducing a problem's instance by the same constant factor on each iteration of the algorithm. In most applications, this constant factor is equal to two.

Eg : Binary Search - In each step, the algorithm divides the search space in half, reducing the problem size by a factor of 2.

4. What is a variable size decrease technique and give an example.

In variable size decrease technique, the size of the problem is reduced by a variable amount in each step.

Example: Quick Sort - The pivot element is chosen dynamically, and the array is partitioned into sub-arrays of varying sizes based on the pivot.

5. Write the time complexity for worst, best and average case of Insertion sort.

$C_{\text{worst}}(n) = O(n^2)$ (when the array is in reverse order)

$C_{\text{best}}(n) = O(n)$ (when the array is already sorted)

$C_{\text{avg}}(n) = O(n^2)$ (quadratic)

6. Give any four comparisons among Depth First Search and Breadth First Search.

	Depth First Search	Breadth First Search
Data structure	stack	Queue
No. of vertex orderings	2 orderings	1 ordering
Edge types	tree and back edges	tree and cross edges
Applications	connectivity, acyclicity, articulation points	connectivity, acyclicity, minimum-edge paths

7. Define digraph and dag.

A directed graph (digraph) is a graph in which edges have a direction, i.e., they are ordered pairs of vertices (u, v) indicating a directed edge from vertex u to vertex v.

A directed acyclic graph (DAG) is a special type of directed graph that does not contain any cycles. In other words, there is no sequence of vertices such that the first vertex is also the last.

8. Write an algorithm to find height of Binary tree.

ALGORITHM *Height(T)*
//Computes recursively the height of a binary tree
//Input: A binary tree *T*
//Output: The height of *T*
if $T = \emptyset$ **return** -1
else return $\max\{Height(T_L), Height(T_R)\} + 1$

9. Write the recurrence relation for Binary tree.

$$A(n(T)) = A(n(T_L)) + A(n(T_R)) + 1 \quad \text{for } n(T) > 0,$$
$$A(0) = 0.$$

10. Define Topological sorting. Give an example.

Topological sorting is a way of arranging the vertices (nodes) of a directed acyclic graph (DAG) in a linear order such that for every directed edge (u, v) from vertex u to vertex v , u comes before v in the ordering. In simpler terms, it's a way of sequencing tasks or events where some tasks depend on others being completed first.

Here is an example of a topological sort:

A $\rightarrow$ B B $\rightarrow$ C C $\rightarrow$ D A $\rightarrow$ D

In this example, task A must be completed before task B can be completed, task B must be completed before task C can be completed, and task C must be completed before task D can be completed. A topological sort of this set of tasks is A, B, C, D.

11. Define following.

a. Tree Edge : A tree edge is an edge that connects two nodes in a tree. Tree edges are directed from a parent node to a child node.

b. Cross Edge : A cross edge is an edge that connects two nodes in a tree that are not parent-child relationships. Cross edges are not directed.

c. Back Edge : A back edge is an edge that connects a node to one of its ancestors in a tree. Back edges are directed.

12. List the Two algorithms for solving topological sorting problem.

- DFS
- Decrease (by one)-and-conquer technique

13. Define decrease-by-one technique in topological sorting.

The decrease-by-one technique is a way to improve the time complexity of topological sorting. This technique is based on the observation that each time a node is topologically sorted, its in-degree decreases by one. By keeping track of the in-degree of each node, we can avoid revisiting nodes that have already been topologically sorted.

14. Define divide and conquer technique and list its steps.

Divide and conquer is a powerful algorithmic technique that involves breaking down a problem into smaller, more manageable subproblems, solving them recursively, and then combining their solutions to solve the original problem.

- Steps:
- (i) Divide: Break the problem into smaller subproblems.
 - (ii) Conquer: Solve the subproblems recursively.
 - (iii) Combine: Combine the solutions of subproblems to solve the original problem.

15. Give the structure of Recurrence equation for Divide and Conquer.

The recurrence equation for divide and conquer is:

$$T(n) = aT(n/b) + c$$

where: $T(n)$ is the time complexity of solving the problem of size n
 a is the number of subproblems into which the problem is divided
 b is the size of each subproblem
 c is the time taken to combine the solutions to the subproblem

16. Write the time complexity for Merge and Quick sort.

The time complexity for Merge Sort is $O(n \log n)$

The time complexity for Quick Sort is

Worst case: $O(n^2)$

Average case: $O(n \log n)$

Best case : $O(n \log n)$

17. Write the time complexity for Strassen's Matrix Multiplication and Multiplication of Larger integer.

The time complexity for Strassen's Matrix Multiplication : $O(n^{\log_2 7})$

The time complexity for Multiplication of Larger integer : $O(n^{\log_2 3})$

18. State Master theorem of Divide and conquer.

THEOREM (Master Theorem) If $f(n) \in \Theta(n^d)$ with $d \geq 0$ in recurrence equation (4.1), then

$$T(n) \in \begin{cases} \Theta(n^d) & \text{if } a < b^d \\ \Theta(n^d \log n) & \text{if } a = b^d \\ \Theta(n^{\log_b a}) & \text{if } a > b^d \end{cases}$$

19. Write the time complexity for worst, best and average case of Binary Search.

$C_{\text{worst}}(n) = O(\log n)$ (Target element found in the last comparison, or not found at all)

$C_{\text{best}}(n) = O(1)$ (Target element found in the middle of the first comparison)

$C_{\text{avg}}(n) = O(\log n)$ (Target element found after a typical number of comparisons)

20. Write the steps followed in divide and Conquer approach.

The steps followed in the divide and conquer approach are:

- (i) Divide the problem into smaller subproblems.
- (ii) Solve the subproblems recursively.
- (iii) Combine the solutions to the subproblems to solve the original problem.

Long Answer Questions

1. Write an algorithm to sort N numbers using Insertion sort. Derive the time complexity for worst case and best case.

ALGORITHM *InsertionSort*($A[0..n-1]$)
//Sorts a given array by insertion sort
//Input: An array $A[0..n-1]$ of n orderable elements
//Output: Array $A[0..n-1]$ sorted in nondecreasing order
for $i \leftarrow 1$ **to** $n-1$ **do**
 $v \leftarrow A[i]$
 $j \leftarrow i-1$
 while $j \geq 0$ **and** $A[j] > v$ **do**
 $A[j+1] \leftarrow A[j]$
 $j \leftarrow j-1$
 $A[j+1] \leftarrow v$

Time Complexity:

The basic operation of the algorithm is the key comparison $A[j] > v$.

The number of key comparisons depends on the nature of the input.

The *worst case* occurs when the array is in reverse order, each element requires a maximum number of comparisons and shifts.

$$C_{\text{worst}}(n) = \sum_{i=1}^{n-1} \sum_{j=0}^{i-1} 1 = \sum_{i=1}^{n-1} i = \frac{(n-1)n}{2} \in \Theta(n^2)$$

The *best case* is when the array is already sorted, only one comparison is made per element.

$$C_{\text{best}}(n) = \sum_{i=1}^{n-1} 1 = n-1 \in \Theta(n).$$

The *average case* is when there is half array comparison in decreasing array.

$$C_{\text{avg}}(n) \approx \frac{n^2}{4} \in \Theta(n^2).$$

2. Define decrease and conquer technique and explain its variations.

Refer 2 marks Q.No. 1,2,3,4

3. Write and explain Depth-First Search Algorithm with example.

DFS is an algorithm for traversing or searching tree or graph data structures. The algorithm starts at the tree root (or some arbitrary node of a graph, sometimes called a 'search key'), and explores as far as possible along each branch before backtracking.

Example: Refer Q.No. 5 b

ALGORITHM DFS(G)

```

//Implements a depth-first search traversal of a given graph
//Input: Graph  $G = \langle V, E \rangle$ 
//Output: Graph  $G$  with its vertices marked with consecutive integers
//in the order they've been first encountered by the DFS traversal
    mark each vertex in  $V$  with 0 as a mark of being "unvisited"
    count  $\leftarrow 0$ 
    for each vertex  $v$  in  $V$  do
        if  $v$  is marked with 0
            dfs( $v$ )

    dfs( $v$ )
    count  $\leftarrow$  count + 1; mark  $v$  with count
    for each vertex  $w$  in  $V$  adjacent to  $v$  do
        if  $w$  is marked with 0
            dfs( $w$ )

```

4. Write and explain Breadth-First Search Algorithm with example.

ALGORITHM BFS(G)

```

//Implements a breadth-first search traversal of a given graph
//Input: Graph  $G = \langle V, E \rangle$ 
//Output: Graph  $G$  with its vertices marked with consecutive integers
//in the order they have been visited by the BFS traversal

    mark each vertex in  $V$  with 0 as a mark of being "unvisited"
    count  $\leftarrow 0$ 
    for each vertex  $v$  in  $V$  do
        if  $v$  is marked with 0
            bfs( $v$ )

    bfs( $v$ )
    count  $\leftarrow$  count + 1; mark  $v$  with count and initialize a queue with  $v$ 
    while the queue is not empty do
        for each vertex  $w$  in  $V$  adjacent to the front vertex do
            if  $w$  is marked with 0
                count  $\leftarrow$  count + 1; mark  $w$  with count
                add  $w$  to the queue
        remove the front vertex from the queue

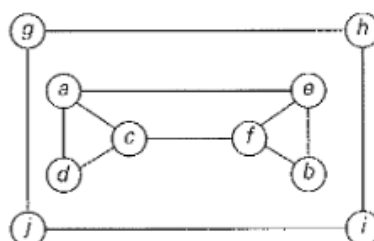
```

BFS is an algorithm for traversing or searching tree or graph data structures. It starts at the tree root (or some arbitrary node of a graph, sometimes called a 'search key'), and explores the neighbor nodes first, before moving to the next level neighbors.

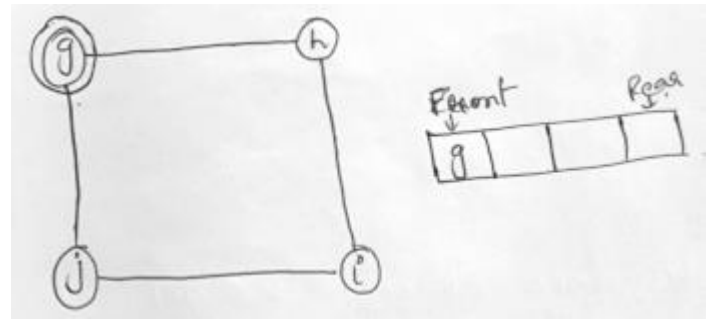
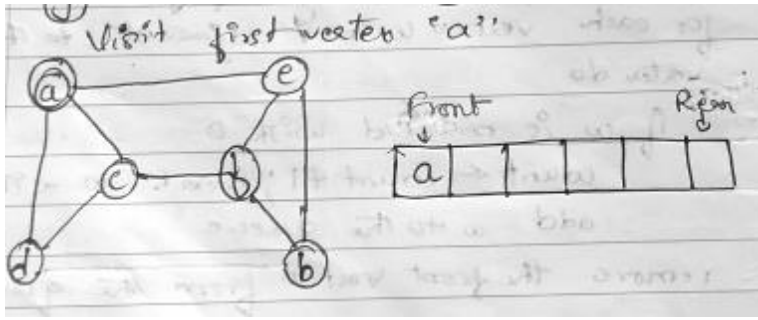
Example : Refer Q.No. 5 a

5. Consider the following graph and perform following traversal and also draw the TREE.

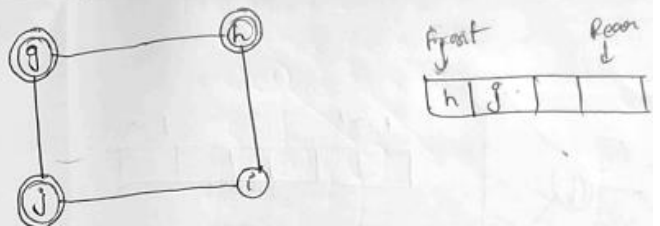
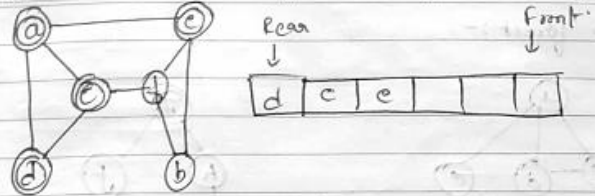
- a. BFS
- b. DFS



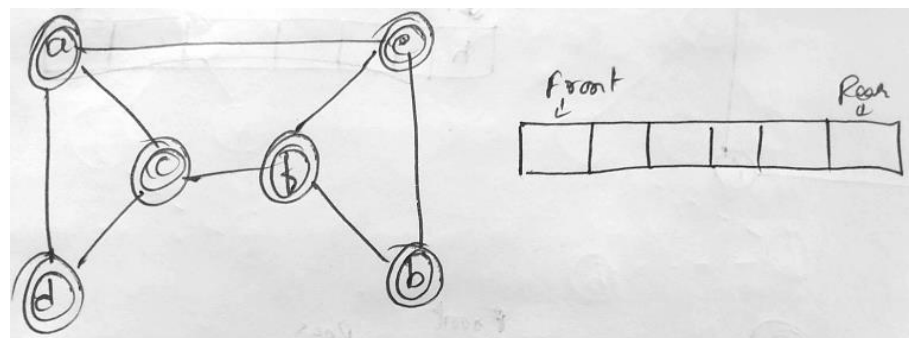
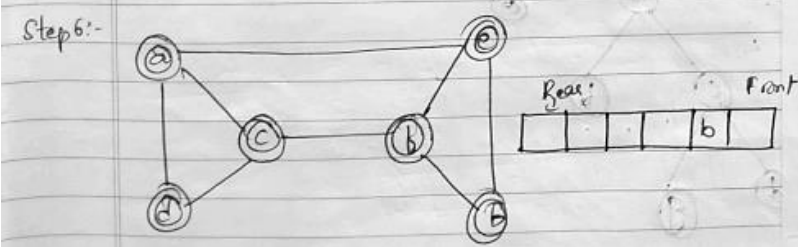
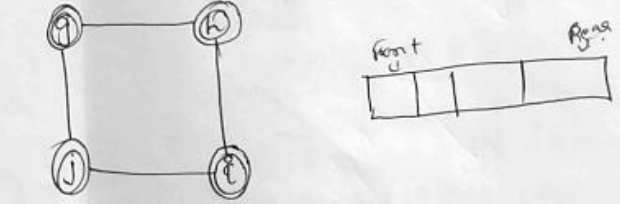
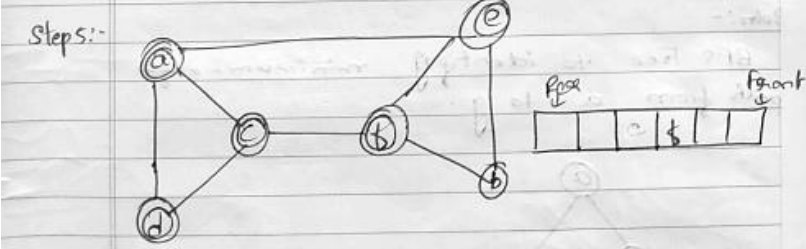
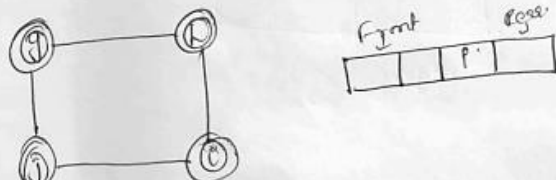
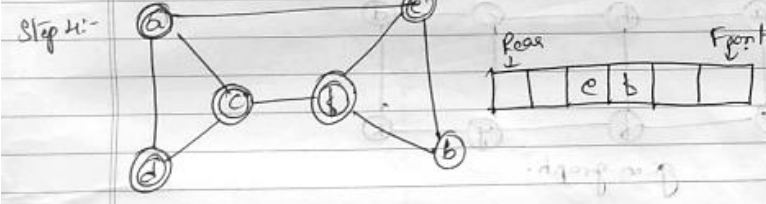
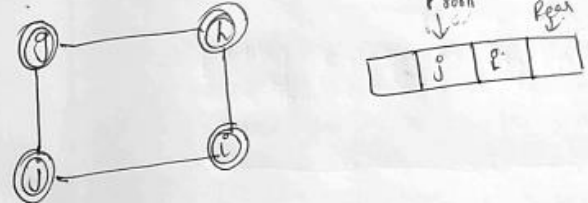
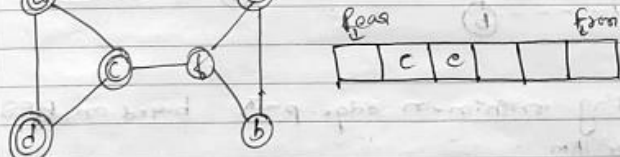
a. BFS



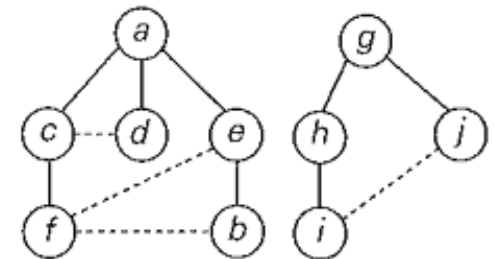
Step 2:- Visit all its adjacency vertices, Remove 'a' from queue.



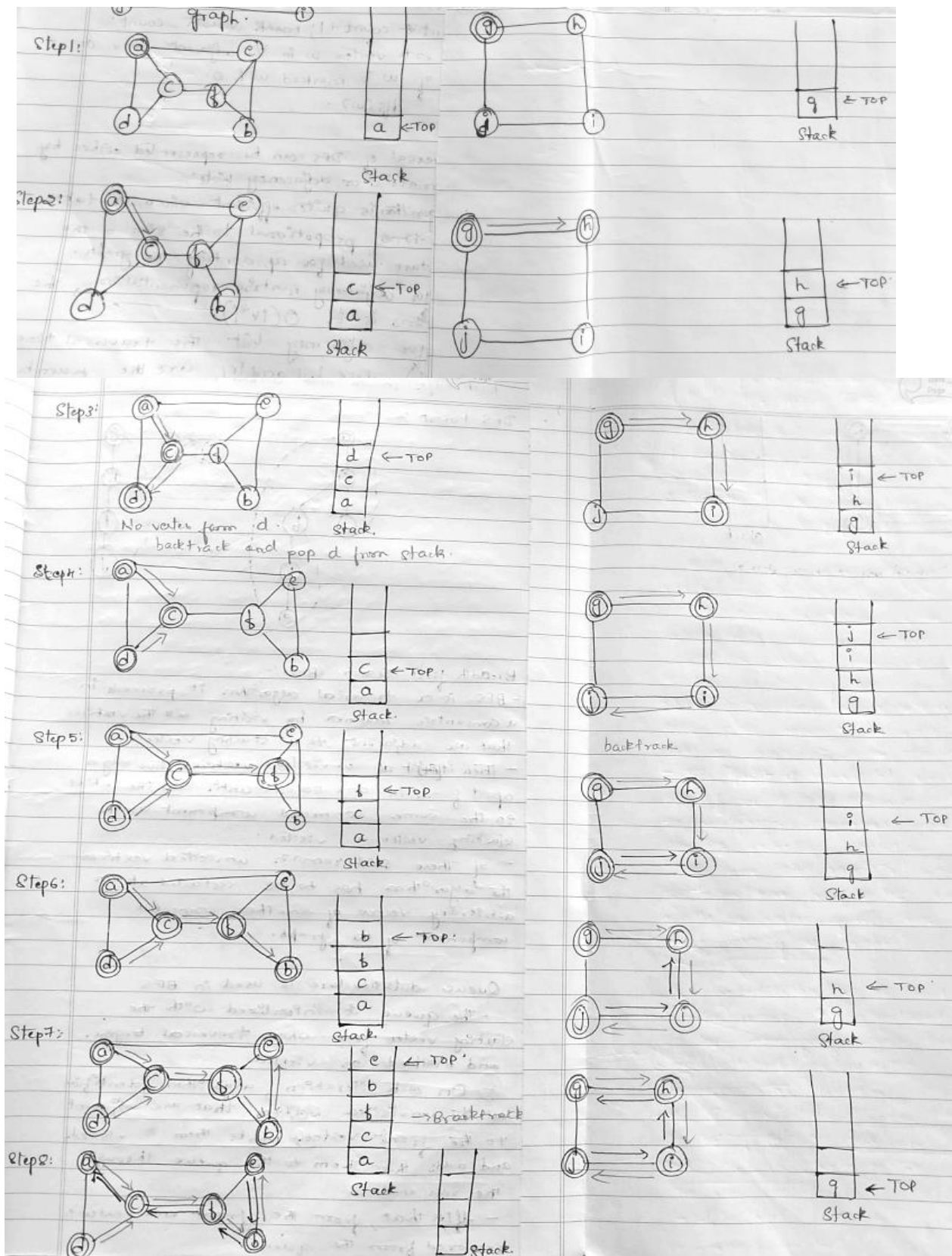
Step 3:- Visit d and all its adjacency vertices if there is no unvisited vertices delete d.



BFS Forest:



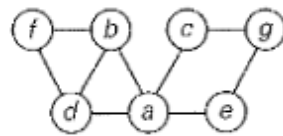
b. DFS



6. Consider the following graph and perform following traversal also draw the TREE.

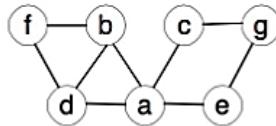
a. BFS

b. DFS



Ans:

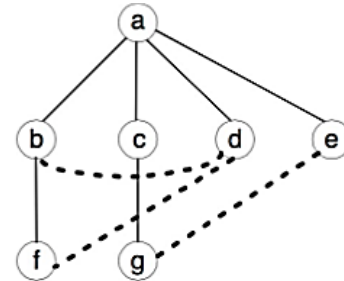
a. BFS



(i)

a b c d e f g

(ii)

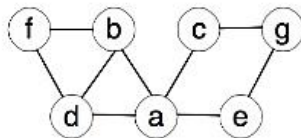


(iii)

(i) the graph; (ii) the traversal's queue; (iii) the tree (the tree and cross edges are shown with solid and dotted lines, respectively).

b. DFS

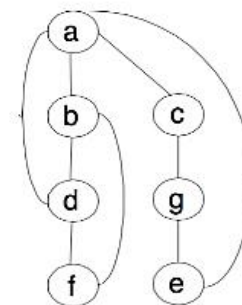
(i) the graph; (ii) the traversal's stack (the first subscript number indicates the order in which the vertex was visited, i.e., pushed onto the stack, the second one indicates the order in which it became a dead-end, i.e., popped off the stack); (iii) the DFS tree (with the tree edges shown with solid lines and the back edges shown with dashed lines).



(i)

f_{4,1} e_{7,4}
d_{3,2} g_{6,5}
b_{2,3} c_{5,6}
a_{1,7}

(ii)



(iii)

7. Give any five comparisons among Depth First Search and Breadth First Search.

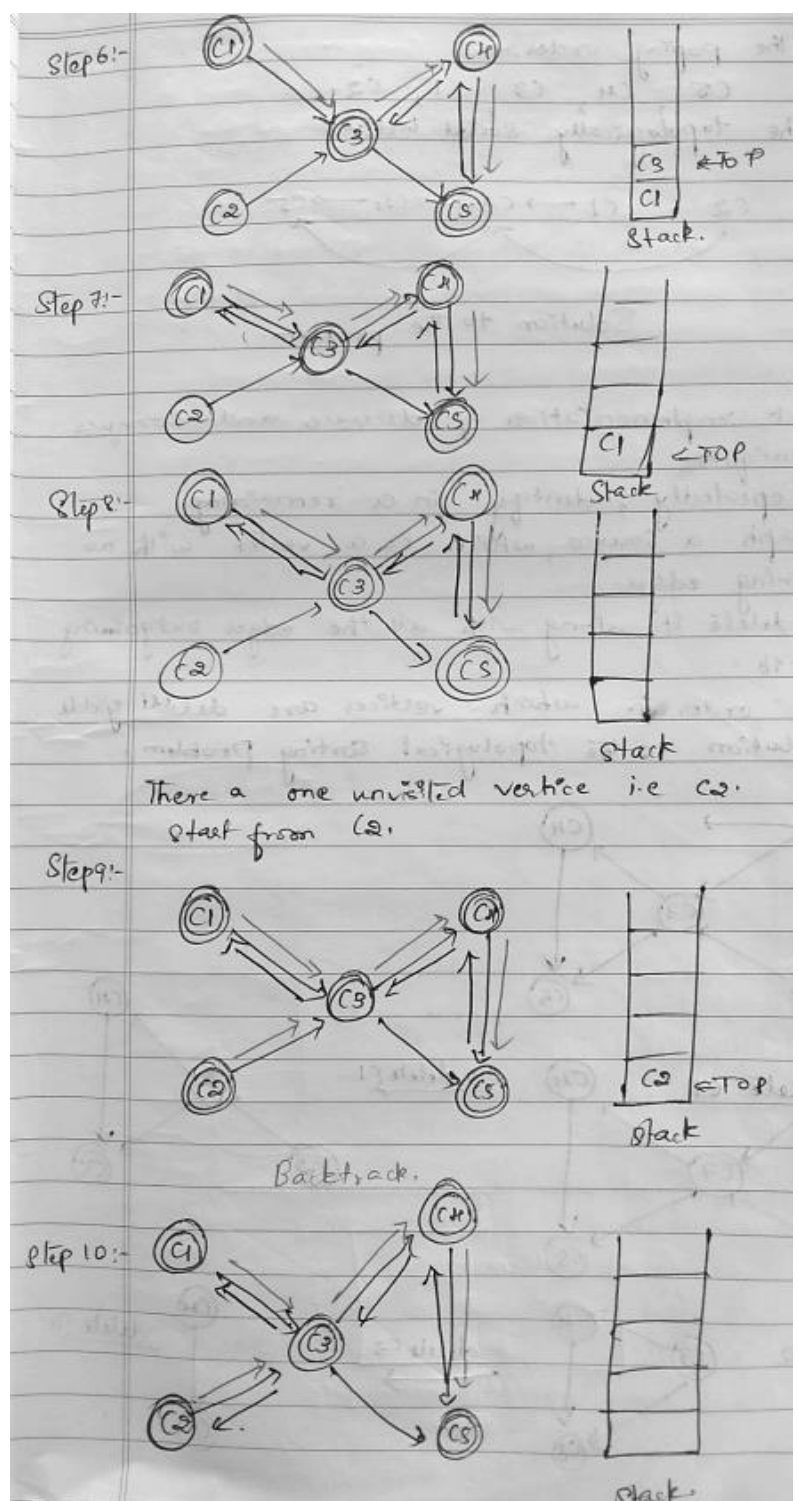
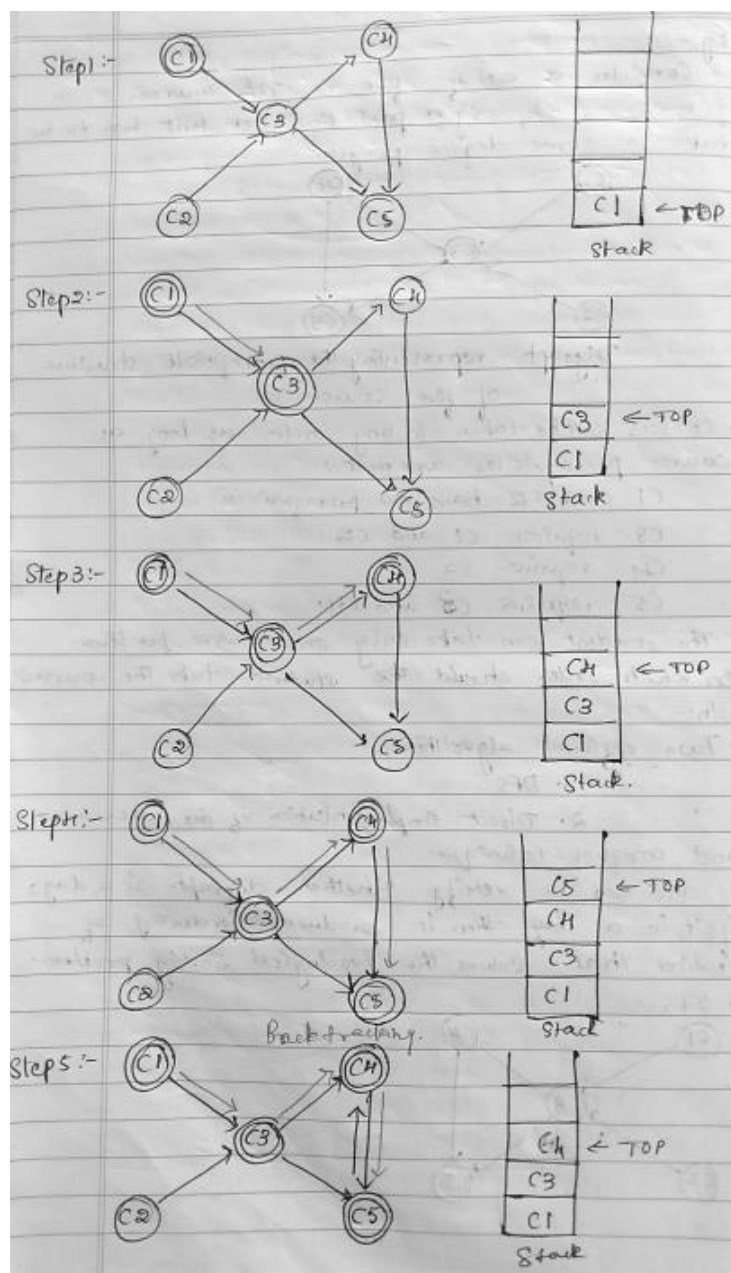
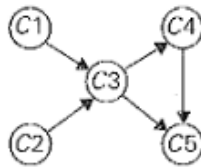
- o **Data Structure:** BFS uses a queue to keep track of the next vertex to visit, whereas DFS uses a stack or can be implemented with recursion.
- o **Traversal Order:** BFS visits all the vertices of a level before going to the next level, while DFS visits a vertex and then iteratively explores its adjacent vertices before backtracking.
- o **Memory Space:** DFS uses less memory than BFS because it's not necessary to store all the child pointers at each level.
- o **Cycle Detection:** DFS is more suitable for cycle detection in a graph because it explores vertices as far as possible before backtracking, while BFS is not suitable for cycle detection.
- o **Path Finding:** BFS is better for path finding problems where the shortest path is required, as it explores vertices in order of their distance from the source, DFS is not suitable for such problems.

8. Write a note on topological sorting with an example.

Topological sorting is a way of arranging the vertices (nodes) of a directed acyclic graph (DAG) in a linear order such that for every directed edge (u, v) from vertex u to vertex v , u comes before v in the ordering. In simpler terms, it's a way of sequencing tasks or events where some tasks depend on others being completed first.

Example : Refer Q.No. 9

9. Explain DFS-based algorithm to solve the topological sorting problem and also write topological order for below graph.

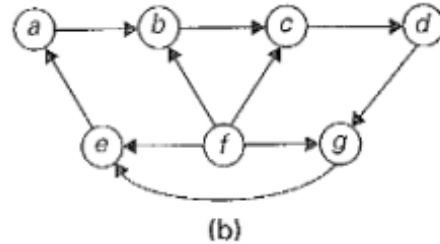
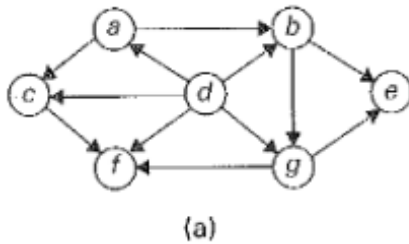


The popping order : C5, C4, C3, C1, C2

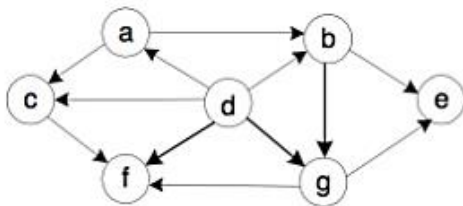
The topologically sorted list :



10. Apply the DFS-based algorithm to solve the topological sorting problem for the following digraphs.



a. The digraph and the stack of its DFS traversal that starts at vertex a are given below:



f
 $e \quad g$
 $b \quad c$
 $a \quad d$

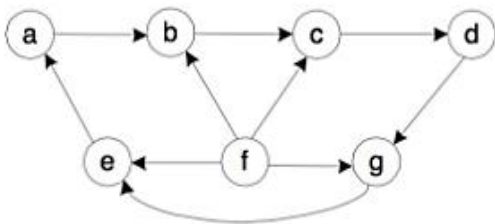
The vertices are popped off the stack in the following order:

$e \ f \ g \ b \ c \ a \ d.$

The topological sorting order obtained by reversing the list above is

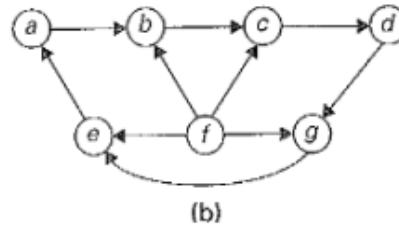
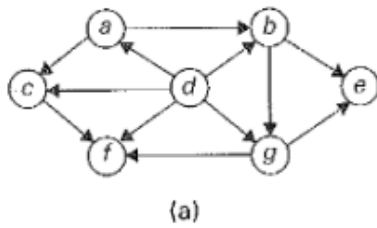
$d \ a \ c \ b \ g \ f \ e.$

b. The digraph below is not a dag. Its DFS traversal that starts at a encounters a back edge from e to a :

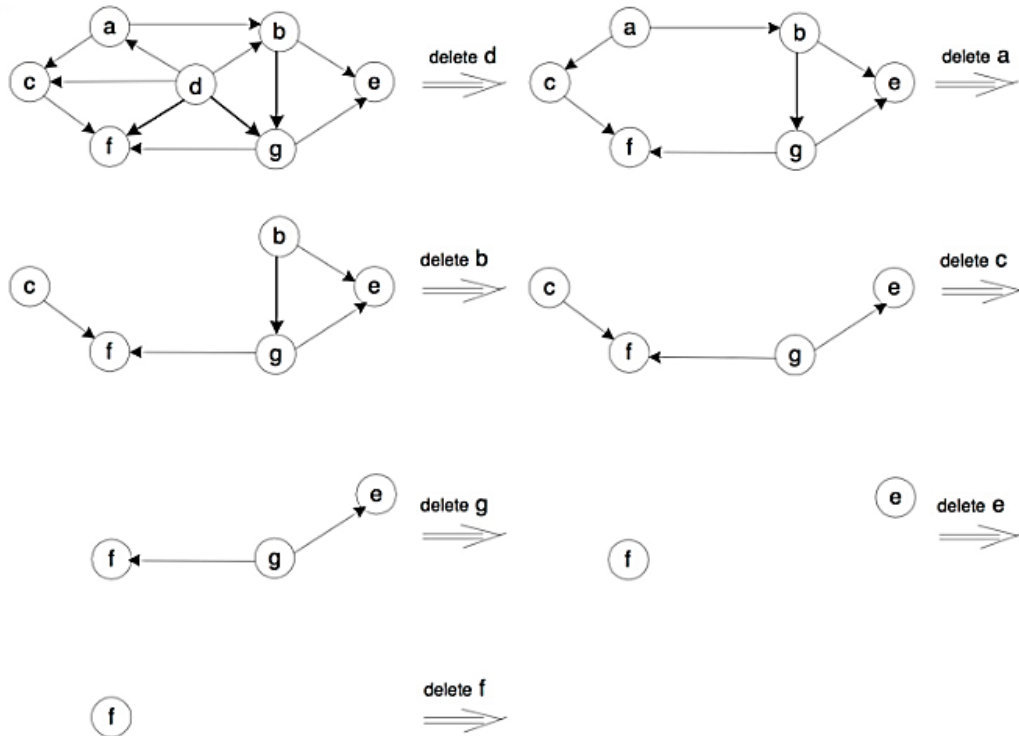


e
 g
 d
 c
 b
 a

11. Apply the source-removal (Decrease by one) algorithm to solve the topological sorting problem for the following digraphs.

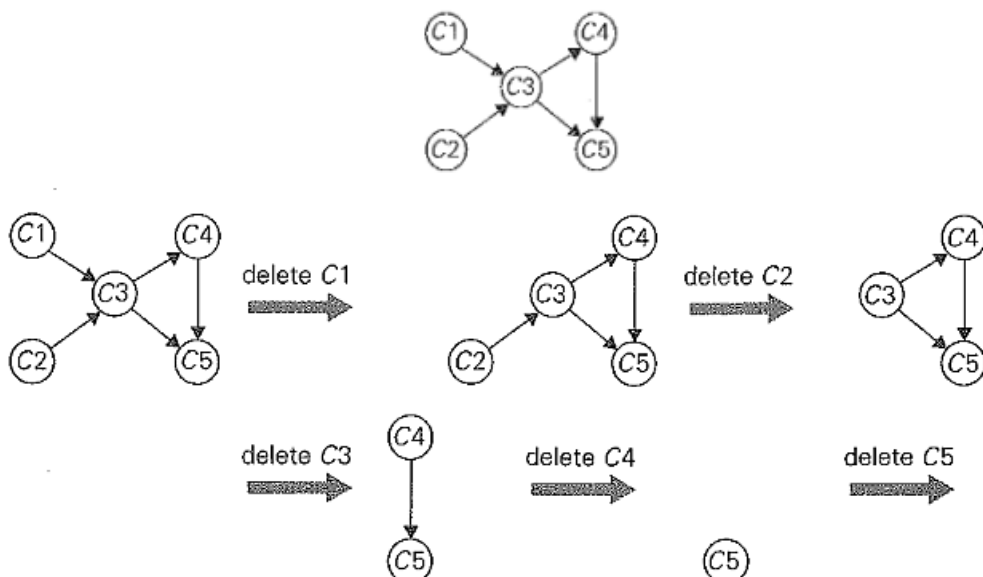


(a)



The topological ordering obtained is $d \ a \ b \ c \ g \ e \ f$.

12. Explain source-removal (Decrease by one) algorithm to solve the topological sorting problem and also write topological order for below graph.



The solution obtained is $C1, C2, C3, C4, C5$

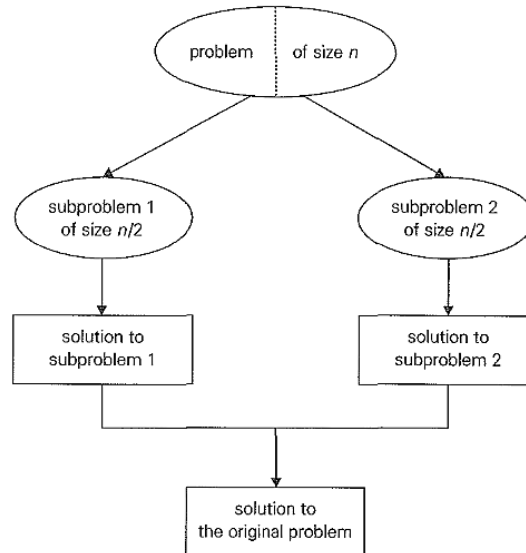
13. Explain divide and conquer technique of solving problem with diagram.

The divide and conquer technique is a problem-solving strategy that involves recursively breaking down a problem into smaller subproblems until they can be solved directly, and then combining the solutions to the subproblems to solve the original problem. This technique is often used to solve problems that are too difficult or time-consuming to solve directly.

Steps: (i) Divide: Break the problem into smaller subproblems.

(ii) Conquer: Solve the subproblems recursively.

(iii) Combine: Combine the solutions of subproblems to solve the original problem.



Examples : Merge Sort, Quick Sort, Binary Search

14. Solve the recursion relation using Master theorems.

$$T(n) = 2T(n/2) + n \quad T(1) = 2$$

Ans: To solve this recurrence using the Master theorem, we first need to identify the values of a , b , and $f(n)$.

- a : The value of a is 2.
- b : The value of b is 2.
- $f(n)$: The value of $f(n)$ is n .

Next, we need to compare $f(n)$ to $n^{\log_b a}$ to determine the appropriate case of the Master theorem.

- $n^{\log_b a}$: $n^{\log_2 2} = n$.

Since $f(n)$ is $O(n)$, which is the same as the worst-case time complexity of the subproblems, we have Case 2 of the Master theorem. This means that the time complexity of the recurrence is $T(n) = O(n \log n)$.

15. Solve the recursion relation using Master theorems.

$$A(n) = 2A(n/2) + 1$$

Thus, for this example, $a = 2$, $b = 2$, and $d = 0$; hence, since $a > b^d$,

$$A(n) \in \Theta(n^{\log_b a}) = \Theta(n^{\log_2 2}) = \Theta(n).$$

16. Write an algorithm to sort N numbers using Merge sort. Derive the time complexity.

ALGORITHM *Mergesort*($A[0..n-1]$)

//Sorts array $A[0..n-1]$ by recursive mergesort
 //Input: An array $A[0..n-1]$ of orderable elements
 //Output: Array $A[0..n-1]$ sorted in nondecreasing order
if $n > 1$
 copy $A[0..[n/2]-1]$ to $B[0..[n/2]-1]$
 copy $A[[n/2]..n-1]$ to $C[0..[n/2]-1]$
 Mergesort($B[0..[n/2]-1]$)
 Mergesort($C[0..[n/2]-1]$)
 Merge(B, C, A)

ALGORITHM *Merge*($B[0..p-1], C[0..q-1], A[0..p+q-1]$)

//Merges two sorted arrays into one sorted array
 //Input: Arrays $B[0..p-1]$ and $C[0..q-1]$ both sorted
 //Output: Sorted array $A[0..p+q-1]$ of the elements of B and C
 $i \leftarrow 0; j \leftarrow 0; k \leftarrow 0$
while $i < p$ **and** $j < q$ **do**
 if $B[i] \leq C[j]$
 $A[k] \leftarrow B[i]; i \leftarrow i + 1$
 else $A[k] \leftarrow C[j]; j \leftarrow j + 1$
 $k \leftarrow k + 1$
if $i = p$
 copy $C[j..q-1]$ to $A[k..p+q-1]$
else copy $B[i..p-1]$ to $A[k..p+q-1]$

Time Complexity:

1. Input size is 'n'.
 2. Time complexity is depends on input size.
 3. Basic operation: Comparison operation.
 The recurrence relation for the number of key comparison $C(n)$ is

$$C(n) = 2C(n/2) + C_{\text{merge}}(n) \quad \text{for } n > 1$$

Time takes to sort both the subarray.
 in merge sort Alg. i.e. $B[0..n/2-1]$ & $C[0..n/2-1]$.

Time taken to merge two individual sorted array.

for $n=1$ then $C(n)=0$
 i.e time complexity is 0
 bcz there is only one element in the array.

Worst case:-
 For worst case $C(n) = n-1$ and the merge
 recurrence relation is,

$$C_{\text{worst}}(n) = 2C_{\text{worst}}(n/2) + n-1 \quad \text{for } n > 1$$

$n-1$ is the number of time taken to merge two sorted array.

$0 \dots n-1 \quad 0 \dots n-1$
 $\swarrow \quad \searrow$
 $0 \dots n-1$

$C_{\text{worst}}(1) = 0$.

While comparing the recurrence relation with Master theorem, the time complexity is

$$C_{\text{worst}}(n) \in (n \log n)$$

17. Write an algorithm to sort N numbers using Quick sort. Derive the time complexity.

ALGORITHM *Quicksort*(A[l..r])
 //Sorts a subarray by quicksort
 //Input: A subarray A[l..r] of A[0..n - 1], defined by its left and right indices
 // l and r
 //Output: Subarray A[l..r] sorted in nondecreasing order
if l < r
 s ← *Partition*(A[l..r]) //s is a split position
 Quicksort(A[l..s - 1])
 Quicksort(A[s + 1..r])

Best Case: If all the splits happen in the middle of corresponding subarrays. The number of key comparisons in the best case will satisfy the recurrence

$$C_{best}(n) = 2C_{best}(n/2) + n \quad \text{for } n > 1, \quad C_{best}(1) = 0.$$

According to the Master Theorem,

$$C_{best}(n) \in \Theta(n \log_2 n)$$

Worst Case : If one of the two subarrays is empty or the size of the other will be less than the size of the subarray being partitioned or the given array is a strictly increasing array.

$$C_{worst}(n) = (n + 1) + n + \dots + 3 = \frac{(n + 1)(n + 2)}{2} - 3 \in \Theta(n^2)$$

Average Case : Let $C_{avg}(n)$ be the average number of key comparisons made by quicksort on a randomly ordered array of size n. Assuming that the partition split can happen in each positions ($0 \leq s \leq n - 1$) with the same probability $1/n$, we get the following recurrence relation.

$$C_{avg}(n) = \frac{1}{n} \sum_{s=0}^{n-1} [(n + 1) + C_{avg}(s) + C_{avg}(n - 1 - s)] \quad \text{for } n > 1,$$

$$C_{avg}(0) = 0, \quad C_{avg}(1) = 0.$$

$$C_{avg}(n) \approx 2n \ln n \approx 1.38n \log_2 n.$$

18. Write an algorithm to find maximum and minimum element in array Derive the time complexity.

- Divide array by calculating mid index i.e. $\text{mid} = l + (r - l)/2$
- Recursively find the maximum and minimum of left part by calling the same function i.e.
 $\text{leftMinMax}[2] = \text{minMax}(X, l, \text{mid})$
- Recursively find the maximum and minimum of right part by calling the same function i.e.
 $\text{rightMinMax}[2] = \text{minMax}(X, \text{mid} + 1, r)$
- Finally, get the overall maximum and minimum by comparing the min and max of both halves.

So the recurrence relation of time complexity is: $T(n) = 2 T(n/2) + 2$, where $T(2) = 1$ and $T(1) = 0$.

Time Complexity : $O(n)$

19. Write an algorithm to search an element in an array of N numbers using Binary search. Derive the time complexity.

ALGORITHM *BinarySearch*($A[0..n-1]$, K)

//Implements nonrecursive binary search

//Input: An array $A[0..n-1]$ sorted in ascending order and

// a search key K

//Output: An index of the array's element that is equal to K

// or -1 if there is no such element

$l \leftarrow 0$; $r \leftarrow n - 1$

while $l \leq r$ **do**

$m \leftarrow \lfloor (l + r)/2 \rfloor$

if $K = A[m]$ **return** m

else if $K < A[m]$ $r \leftarrow m - 1$

else $l \leftarrow m + 1$

return -1

Time Complexity:

Basic Operation : Comparison Operation

The number of number comparison depends not only on 'n' but also search element location.

Best Case: Search key is present at the middle of the array. It requires only one comparison.

$$C_{\text{best}}(n) = \Omega(1)$$

Worst Case : Maximum Number of comparison operation is required. The recurrence relation is

$$C_{\text{worst}}(n) = C_{\text{worst}}(\lfloor n/2 \rfloor) + 1 \quad \text{for } n > 1, \quad C_{\text{worst}}(1) = 1$$

Assume $n=2^k$

$$\begin{aligned} C_{\text{worst}}(2^k) &= C_{\text{worst}}(2^{k/2}) + 1 \\ &= C_{\text{worst}}(2^{k-1}) + 1 \\ &= C_{\text{worst}}(2^{k-2}) + 1 + 1 \\ &= C_{\text{worst}}(2^{k-2}) + 2 \\ &\dots\dots\dots \\ &= C_{\text{worst}}(2^{k-i}) + i \end{aligned}$$

If $i=k$,

$$C_{\text{worst}}(n) = C_{\text{worst}}(2^{k-k}) + k = k$$

Let $k=\log_2 n$

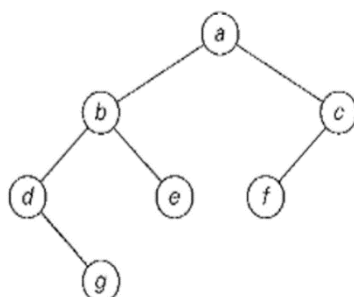
$$C_{\text{worst}}(n) = \Theta(\log_2 n)$$

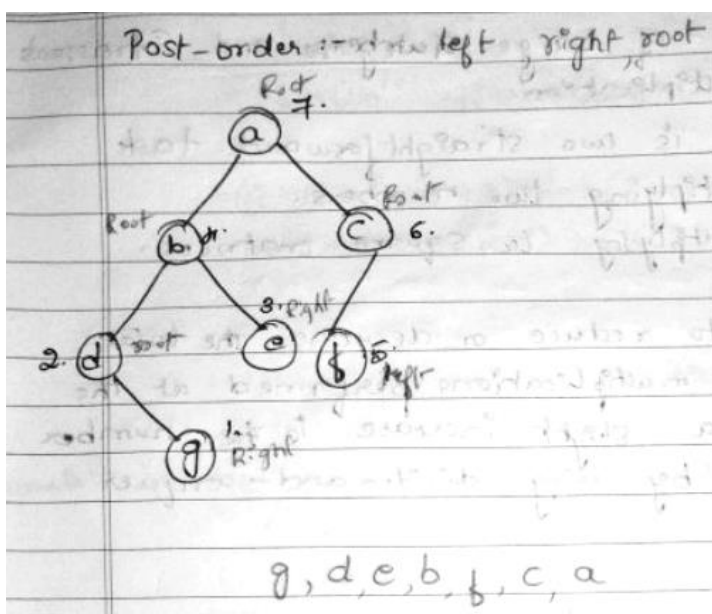
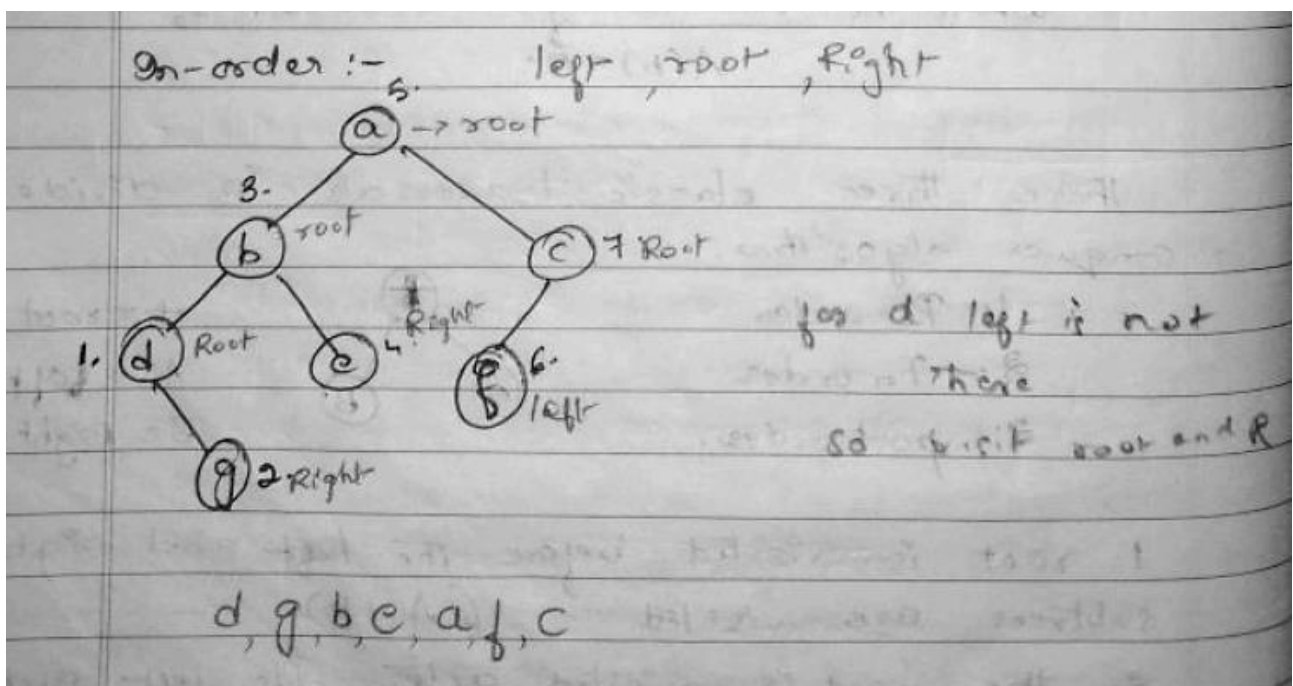
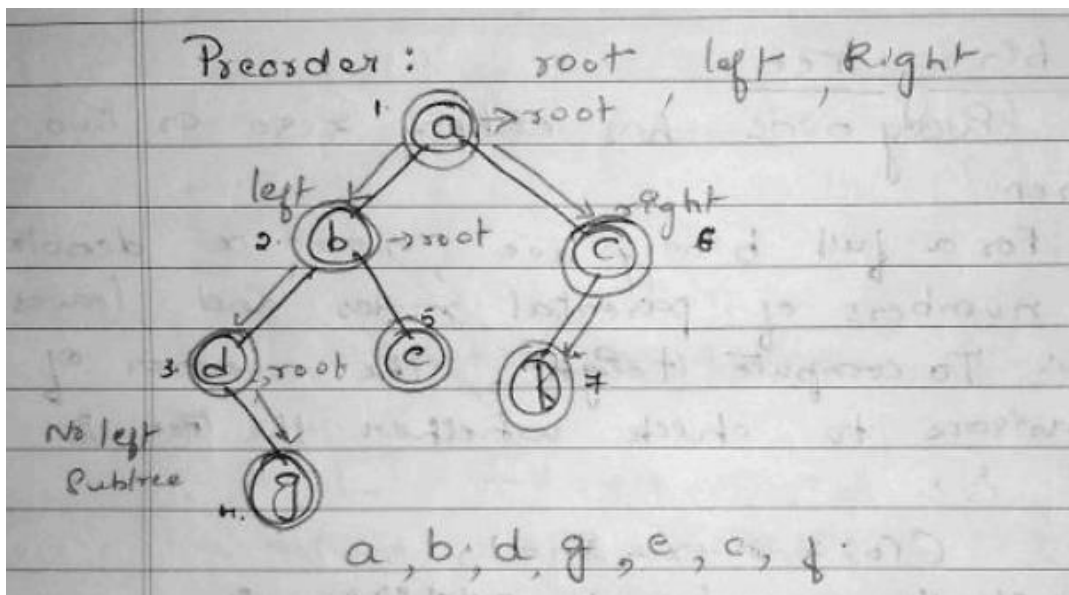
Average Case : For succesful search, $C_{\text{avg}}(n) = \log_2 n - 1$

For unsuccessful search, $C_{\text{avg}}(n) = \log_2 (n+1)$

20. Traverse the Binary Tree

a)Inorder b)Preorder c)Postorder





21. Explain the Strassen's algorithm of matrix multiplication and derive the time complexity.

The principal insight of the algorithm lies in the discovery that we can find the product C of two 2-by-2 matrices A and B with just seven multiplications. This is accomplished by using the following formulas:

$$\begin{bmatrix} c_{00} & c_{01} \\ c_{10} & c_{11} \end{bmatrix} = \begin{bmatrix} a_{00} & a_{01} \\ a_{10} & a_{11} \end{bmatrix} * \begin{bmatrix} b_{00} & b_{01} \\ b_{10} & b_{11} \end{bmatrix}$$

$$= \begin{bmatrix} m_1 + m_4 - m_5 + m_7 & m_3 + m_5 \\ m_2 + m_4 & m_1 + m_3 - m_2 + m_6 \end{bmatrix}$$

where

$$m_1 = (a_{00} + a_{11}) * (b_{00} + b_{11})$$

$$m_2 = (a_{10} + a_{11}) * b_{00}$$

$$m_3 = a_{00} * (b_{01} - b_{11})$$

$$m_4 = a_{11} * (b_{10} - b_{00})$$

$$m_5 = (a_{00} + a_{01}) * b_{11}$$

$$m_6 = (a_{10} - a_{00}) * (b_{00} + b_{01})$$

$$m_7 = (a_{01} - a_{11}) * (b_{10} + b_{11}).$$

If $M(n)$ is the number of multiplications made by Strassen's algorithm in multiplying two n -by- n matrices (where n is a power of 2), we get the following recurrence relation for it:

$$M(n) = 7M(n/2) \text{ for } n > 1, \quad M(1) = 1$$

Since $n = 2^k$

$$\begin{aligned} M(2^k) &= 7M(2^{k-1}) = 7[7M(2^{k-2})] = 7^2M(2^{k-2}) = \dots \\ &= 7^i M(2^{k-i}) \dots = 7^k M(2^{k-k}) = 7^k \end{aligned}$$

Since $k = \log_2 n$,

$$M(n) = 7^{\log_2 n} = n^{\log_2 7} \approx n^{2.807}$$

22. Apply Strassen's algorithm to multiply two Matrixes.

$$\begin{bmatrix} 3 & -1 \\ 2 & 4 \end{bmatrix} * \begin{bmatrix} 2 & 3 \\ -1 & 4 \end{bmatrix}$$

23. Compute 1234×2526 using divide and conquer approach for the multiplication of two large numbers.

24. Explain the Multiplication of two larger integer using divide and conquer approach and derive the time complexity.

The divide and conquer approach for multiplication of two large integers is a recursive algorithm that breaks down the problem into smaller subproblems until they can be solved directly, and then combines the solutions to the subproblems to solve the original problem.

For any pair of two-digit integers $a = a_1a_0$ and $b = b_1b_0$, their product c can be computed by the formula

$$c = a * b = c_210^2 + c_110^1 + c_0,$$

where

$c_2 = a_1 * b_1$ is the product of their first digits,

$c_0 = a_0 * b_0$ is the product of their second digits,

$c_1 = (a_1 + a_0) * (b_1 + b_0) - (c_2 + c_0)$ is the product of the sum of the a 's digits and the sum of the b 's digits minus the sum of c_2 and c_0 .

For two n -digit integers a and b ,

$$\begin{aligned} c &= a * b = (a_110^{n/2} + a_0) * (b_110^{n/2} + b_0) \\ &= (a_1 * b_1)10^n + (a_1 * b_0 + a_0 * b_1)10^{n/2} + (a_0 * b_0) \\ &= c_210^n + c_110^{n/2} + c_0, \end{aligned}$$

where

$c_2 = a_1 * b_1$ is the product of their first halves,

$c_0 = a_0 * b_0$ is the product of their second halves,

$c_1 = (a_1 + a_0) * (b_1 + b_0) - (c_2 + c_0)$ is the product of the sum of the a 's halves and the sum of the b 's halves minus the sum of c_2 and c_0 .

Since multiplication of n -digit numbers requires three multiplications of $n/2$ -digit numbers, the recurrence for the number of multiplications $M(n)$ will be,

$$M(n) = 3M(n/2) \quad \text{for } n > 1, \quad M(1) = 1.$$

Solving it by backward substitutions for $n = 2^k$ yields

$$\begin{aligned} M(2^k) &= 3M(2^{k-1}) = 3[3M(2^{k-2})] = 3^2 M(2^{k-2}) \\ &= \dots = 3^i M(2^{k-i}) = \dots = 3^k M(2^{k-k}) = 3^k. \end{aligned}$$

Since $k = \log_2 n$,

$$M(n) = 3^{\log_2 n} = n^{\log_2 3} \approx n^{1.585}.$$

25. Compute 34×26 using divide and conquer approach for the multiplication of two large numbers.

26. Apply Strassen's algorithm to compute exiting the recursion when $n = 2$, i.e., computing the products of 2-by-2 matrices by the brute-force algorithm.

$$\begin{bmatrix} 1 & 0 & 2 & 1 \\ 4 & 1 & 1 & 0 \\ 0 & 1 & 3 & 0 \\ 5 & 0 & 2 & 1 \end{bmatrix} \times \begin{bmatrix} 0 & 1 & 0 & 1 \\ 2 & 1 & 0 & 4 \\ 2 & 0 & 1 & 1 \\ 1 & 3 & 5 & 0 \end{bmatrix}$$